SalesCRM Agent App — API Reference

Entity search · Entity contacts by entity · Combined visit creation (entity + contact + visit)

Base URL prefix: `/api/v1`
Auth: All three endpoints require a valid JWT in the request header: `Authorization: Bearer <token>`, and the token must belong to an **AGENT** role user.
Scope: These are agent-app-only endpoints (under `/agent/...`) — results are automatically restricted to the agent's assigned states where applicable.

1. Search Entities by Name

`GET /api/v1/agent/entity/search`

Fast, paginated, type-ahead style search over entities (schools/hospitals) by name. Returns only `id` and `name` — use this to populate a search/picker list, then use the picked entity's id as `entityId` in API 3.

Query Parameters

Field	Type	Required	Notes
`name`	string	No	Prefix match, case-insensitive (e.g. "st mar" matches "St Mary's School"). Omit to list all.
`page`	number	No	Default 1.
`limit`	number	No	Default 10.

Behavior Notes

- Match is anchored at the start of the name (prefix search) and uses an index on `name` for speed — it is not a "contains anywhere" search.
- Response is automatically scoped to the agent's assigned states — no need to pass state filters.

Example Request

```
GET /api/v1/agent/entity/search?name=st+mar&page=1&limit=10
Authorization: Bearer <token>
```

Example Response — 200 OK


```
{
  "docs": [
    {
      "_id": "656f1a2b3c4d5e6f7a8b9c0d",
      "name": "St Mary's School",
      "id": "656f1a2b3c4d5e6f7a8b9c0d"
    },
    {
      "_id": "656f1a2b3c4d5e6f7a8b9c0e",
      "name": "St Martin's Hospital",
      "id": "656f1a2b3c4d5e6f7a8b9c0e"
    }
  ],
  "totalDocs": 2,
  "limit": 10,
  "page": 1,
  "totalPages": 1,
  "hasNextPage": false,
  "hasPrevPage": false,
```

```
  "nextPage": null,
  "prevPage": null,
  "pagingCounter": 1
}
```

2. Get All Contacts of an Entity

`GET /api/v1/agent/entity-contact/by-entity`

After an entity is picked (via API 1 or otherwise), call this to list all of its contacts, so the agent can either select an existing contact or choose to add a new one for API 3.

Query Parameters

Field	Type	Required	Notes
`entityId`	string	Yes	The entity's _id. Returns 400 if missing.

Example Request

```
GET /api/v1/agent/entity-contact/by-entity?entityId=656f1a2b3c4d5e6f7a8b9c0d
Authorization: Bearer <token>
```

Example Response — 200 OK

```
[
  {
    "_id": "657a1b2c3d4e5f60718293a4",
    "name": "Dr. Ramesh Gupta",
    "contact": "9876543210",
    "email": "ramesh@stmarys.edu",
    "role": "MEDICAL_DIRECTOR",
    "roleDesc": null,
    "entityId": "656f1a2b3c4d5e6f7a8b9c0d",
    "createdBy": "656a0011223344556677889a",
    "createdAt": "2026-09-10T08:12:00.000Z"
  }
]
```

Notes

- Returns a plain array (not paginated) — an entity is expected to have a small, bounded number of contacts.
- `role` is one of: MEDICAL_DIRECTOR, ADMINISTRATOR, DOCTOR, NURSE, OTHER.

3. Create Visit (with Entity & Contact resolution)

`POST /api/v1/agent/visit/with-entity`

Single call used by the "Add Visit" screen. Depending on what the agent searched/selected, it will either re-use an existing entity/contact, or create a new one — then creates the visit. **Entity, entity-contact and visit are written in one database transaction: if any step fails, none of the three documents are saved** (no partial/mismatched data).

Content-Type

`multipart/form-data` (required — the `selfie` is a file). Nested objects (`entity`, `entityContact`, `location`) must be sent using bracket notation field names, e.g. `entity[name]`, `location[latitude]`, `entityContact[contact]`. This is the same convention already used by the existing plain visit-create and entity-photo-upload endpoints.

The App Flow (for context)

- Agent searches entity (API 1). Found it → capture `entityId`. Not found → collect new entity details (name required, rest optional).
- If an entity was found, fetch its contacts (API 2) and let the agent pick one (→ `entityContactId`), or add a new one (name & contact required).
- Capture the agent's current GPS (`location`) and take a `selfie` photo — both required for every visit.
- Submit everything to this one endpoint.

Form Fields

Field	Type	Required	Notes
`entityId`	string	Conditional*	Existing entity id. Send this OR entity — never both.
`entity[name]`	string	Conditional*	Required only when creating a new entity (entityId not sent).
`entity[isActive]`	boolean	No	Optional, saved if sent.
`entity[hospitalType]`	string	No	Optional, saved if sent.
`entity[bedCapacity]`	number	No	Optional, saved if sent.
`entity[address][...]`	string/number	No	pincode, state, district, city, streetAddress — all optional.
`entity[location][latitude/longitude]`	string	No	Optional. If omitted, defaults to this visit's GPS (below).
`entityContactId`	string	Conditional*	Existing contact id. Send this OR entityContact — never both. Must belong to the resolved entity.
`entityContact[name]`	string	Conditional*	Required only when creating a new contact (entityContactId not sent).
`entityContact[contact]`	string	Conditional*	Phone number. Required only when creating a new contact.
`entityContact[email]`	string	No	Optional, saved if sent.
`entityContact[role]`	string	No	One of MEDICAL_DIRECTOR, ADMINISTRATOR, DOCTOR, NURSE, OTHER. Optional.
`entityContact[roleDesc]`	string	No	Optional, saved if sent.
`location[latitude]`	string	Yes	Agent's current GPS latitude at time of visit.
`location[longitude]`	string	Yes	Agent's current GPS longitude at time of visit.
`location[accuracy]`	string	No	GPS accuracy in meters, if available.
`selfie`	file	Yes	Image file (multipart file field). Request is rejected if missing.

Field	Type	Required	Notes
`remark`	string	No	Optional, saved if sent.
`worklogId`	string	No	Optional, saved if sent.

** Conditional fields: exactly one side of each either/or pair must be sent — sending both, or neither, is rejected with a 400 before anything is written to the database.*

Do **not** send `visitedAt` or `userId` — the server always sets these itself (server time, and the authenticated agent from the token).

Validation Rules (checked before any write)

- Exactly one of `entityId` / `entity.name` must be present.
- Exactly one of `entityContactId` / (`entityContact.name` & `entityContact.contact`) must be present.
- `location.latitude` and `location.longitude` are required.
- `selfie` file is required.
- If `entityContactId` is sent, it is verified to belong to the resolved entity — otherwise rejected.

Error Responses — 400 Bad Request

message
`Send either entityId or entity, not both`
`entityId or entity.name is required`
`Send either entityContactId or entityContact, not both`
`entityContactId or entityContact.name & entityContact.contact are required`
`location.latitude and location.longitude are required`
`selfie is required`
`Entity not found for given entityId`
`Entity contact not found for given entityContactId`
`entityContactId does not belong to the resolved entity`

Example Request (multipart fields)

```
POST /api/v1/agent/visit/with-entity
Authorization: Bearer <token>
Content-Type: multipart/form-data

entity[name]            = "St Mary's School"
entity[address][state]  = "Uttar Pradesh"
entity[address][city]   = "Lucknow"
entityContact[name]     = "Dr. Ramesh Gupta"
entityContact[contact]  = "9876543210"
entityContact[role]     = "MEDICAL_DIRECTOR"
location[latitude]      = "26.8467"
location[longitude]     = "80.9462"
location[accuracy]      = "12"
remark                  = "First visit, discussed onboarding"
selfie                  = <binary image file>
```

Example Response — 200/201

```
{
  "entity": {
    "_id": "656f1a2b3c4d5e6f7a8b9c0d",
    "name": "St Mary's School",
    "address": { "state": "Uttar Pradesh", "city": "Lucknow" },
    "createdBy": "656a001122334455667889a",
```

```
    "createdAt": "2026-10-29T10:15:00.000Z"
  },
  "entityContact": {
    "_id": "657a1b2c3d4e5f60718293a4",
    "name": "Dr. Ramesh Gupta",
    "contact": "9876543210",
    "role": "MEDICAL_DIRECTOR",
    "entityId": "656f1a2b3c4d5e6f7a8b9c0d",
    "createdBy": "656a0011223344556677889a"
  },
  "visit": {
    "_id": "657a2b3c4d5e6f7081920a3b",
    "visitedAt": "2026-10-29T10:15:00.000Z",
    "entityId": "656f1a2b3c4d5e6f7a8b9c0d",
    "entityContactId": "657a1b2c3d4e5f60718293a4",
    "userId": "656a0011223344556677889a",
    "remark": "First visit, discussed onboarding",
    "location": "657a2b1a2b3c4d5e6f708192",
    "selfie": {
      "url": "https://<bucket>.s3.amazonaws.com/crm/visit/<visitId>.jpg",
      "fileName": "<visitId>.jpg",
      "fileType": "image/jpeg",
      "size": 184320,
      "uploadTime": "2026-10-29T10:15:02.000Z"
    }
  }
}
```


Note: the selfie upload happens right after the database transaction commits. In the rare case the S3 upload itself fails after a successful save, the response's `visit.selfie` will be absent even though entity/contact/visit were saved — treat a missing `visit.selfie` in the response as a signal to retry the upload for that visit id if this matters for your flow.